

Java Pro

Java 进阶

▼ 注意

以下内容均基于 Java JDK 8 版本编写，不排除在更高版本中有部分改动的可能性。

更高速的输入输出

`Scanner` 和 `System.out.print` 在最开始会工作得很好，但是在处理更大的输入的时候会降低效率，因此我们会需要使用一些方法来提高 IO 速度。

使用 Kattio + StringTokenizer 作为输入

最常用的方法之一是使用来自 Kattis 的 [Kattio.java](#) 来提高 IO 效率。¹这个方法会将 `StringTokenizer` 与 `PrintWriter` 包装在一个类中方便使用。而在具体进行解题的时候（假如赛会/组织方允许）可以直接使用这个模板。

下方即为应包含在代码中的 IO 模板，由于 Kattis 的原 Kattio 包含一些并不常用的功能，下方的模板经过了一些调整（原 Kattio 使用 MIT 作为协议）。

```
1 class Kattio extends PrintWriter {
2     private BufferedReader r;
3     private StringTokenizer st;
4     // 标准 IO
5     public Kattio() { this(System.in, System.out); }
6     public Kattio(InputStream i, OutputStream o) {
7         super(o);
8         r = new BufferedReader(new InputStreamReader(i));
9     }
10    // 文件 IO
11    public Kattio(String input, String output) throws IOException {
12        super(output);
13        r = new BufferedReader(new FileReader(input));
14    }
15    // 在没有任何其他输入时返回 null
16    public String next() {
17        try {
18            while (st == null || !st.hasMoreTokens())
19                st = new StringTokenizer(r.readLine());
20            return st.nextToken();
21        } catch (Exception e) {}
22        return null;
23    }
24    public int nextInt() { return Integer.parseInt(next()); }
25    public double nextDouble() { return Double.parseDouble(next()); }
26    public long nextLong() { return Long.parseLong(next()); }
27 }
```

而下方代码简单展示了 Kattio 的使用：

```
1 class Test {
2     public static void main(String[] args) {
3         Kattio io = new Kattio();
4         // 字符串输入
5         String str = io.next();
6         // int 输入
7         int num = io.nextInt();
8         // 输出
9         io.println("Result");
10        // 请确保关闭 IO 流以确保输出被正确写入
11        io.close();
12    }
13 }
```

使用 StreamTokenizer 作为输入

在某些情况使用 `StringTokenizer` 会导致 MLE（Memory Limit Exceeded，超过内存上限），此时我们需要使用 `StreamTokenizer` 作为输入。

```
1 import java.io.*;
2 public class Main {
3     // IO 代码
4     public static StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in), 32768));
5     public static PrintWriter out = new PrintWriter(new OutputStreamWriter(System.out));
6     public static double nextDouble() throws IOException { in.nextToken(); return in.nval; }
7     public static float nextFloat() throws IOException { in.nextToken(); return (float)in.nval; }
8     public static int nextInt() throws IOException { in.nextToken(); return (int)in.nval; }
9     public static String next() throws IOException { in.nextToken(); return in.sval; }
10    public static long nextLong() throws Exception { in.nextToken(); return (long)in.nval; }
11
12    // 使用示例
13    public static void main(String[] args) throws Exception {
14        int n = nextInt();
15        out.println(n);
16        out.close();
17    }
18 }
```

Kattio + StringTokenizer 的方法与 StreamTokenizer 的方法之间的分析与对比

- 1. StreamTokenizer 相较于 StringTokenizer 使用的内存较少，当 Java 标程 MLE 时可以尝试使用 StreamTokenizer，但是 StreamTokenizer 会丢失精度，读入部分数据时会出现问题；
 - StreamTokenizer 源码存在 Type，该 Type 根据你输入内容来决定类型，倘若你输入类似于 1230i 以数字开头的字符串，他会强制认为你的类型是 double 类型，因此在读入中以 double 类型去读 String 类型便会抛出异常；
 - StreamTokenizer 在读入 1e14 以上大小的数字会丢失精度；
- 2. 在使用 PrintWriter 情况下，需注意在程序结束最后 close() 关闭输出流或在需要输出的时候使用 flush() 清除缓冲区，否则内容将不会被写入到控制台/文件中。
- 3. Kattio 是继承自 PrintWriter 类，自身对象具有了 PrintWriter 的功能，因此可以直接调用 PrintWriter 类的函数输出，同时将 StringTokenizer 作为了自身的成员变量来修改。而第二种 Main 是同时将 StreamTokenizer 与 PrintWriter 作为了自身的成员变量，因此在使用上有些许差距。

综上所述，在大部分情况下，StringTokenizer 的使用处境要优越于 StreamTokenizer，在极端 MLE 的情况下可以尝试 StreamTokenizer，同时 int 范围以上的数据 StreamTokenizer 处理是无能为力的。

BigInteger 与数论

BigInteger 是 Java 提供的高精度计算类，可以很方便地解决高精度问题。

初始化

BigInteger 常用创建方式有如下二种：

```
1 import java.io.PrintWriter;
2 import java.math.BigInteger;
3
4 class Main {
5     static PrintWriter out = new PrintWriter(System.out);
6     public static void main(String[] args) {
7         BigInteger a = new BigInteger("12345678910"); // 将字符串以十进制的形式创建 BigInteger 对象
8         out.println(a); // a 的值为 12345678910
9         BigInteger b = new BigInteger("1E", 16); // 将字符串以指定进制的形式创建 BigInteger 对象
10        out.println(b); // b 的值为 30
11        out.close();
12    }
13 }
```

基本运算

以下均用 this 代替当前 BigInteger：

函数名	功能
abs()	返回 this 的绝对值
negate()	返回 - this
add(BigInteger val)	返回 this + val
subtract(BigInteger val)	返回 this - val
multiply(BigInteger val)	返回 this * val
divide(BigInteger val)	返回 this / val
remainder(BigInteger val)	返回 this % val
mod(BigInteger val)	返回 this mod val
pow(int e)	返回
and(BigInteger val)	返回 this & val
or(BigInteger val)	返回 this val
not()	返回 ~ this
xor(BigInteger val)	返回 this ^ val
shiftLeft(int n)	返回 this << n
shiftRight(int n)	返回 this >> n
max(BigInteger val)	返回 this 与 val 的较大值
min(BigInteger val)	返回 this 与 val 的较小值
bitCount()	返回 this 的二进制中不包括符号位的 1 的个数
bitLength()	返回 this 的二进制中不包括符号位的长度
getLowestSetBit()	返回 this 的二进制中最右边的位置
compareTo(BigInteger val)	比较 this 和 val 值大小
toString()	返回 this 的 10 进制字符串表示形式
toString(int radix)。	返回 this 的 raidx 进制字符串表示形式

使用案例如下：

```
1 import java.io.PrintWriter;
2 import java.math.BigInteger;
3
4 public class Main {
5     static PrintWriter out = new PrintWriter(System.out);
6     static BigInteger a, b;
7
8     static void abs() {
9         out.println("abs:");
10        a = new BigInteger("-123");
11        out.println(a.abs()); // 输出 123
12        a = new BigInteger("123");
```

```

13     out.println(a.abs()); // 输出 123
14 }
15
16 static void negate() {
17     out.println("negate:");
18     a = new BigInteger("-123");
19     out.println(a.negate()); // 输出 123
20     a = new BigInteger("123");
21     out.println(a.negate()); // 输出 -123
22 }
23
24 static void add() {
25     out.println("add:");
26     a = new BigInteger("123");
27     b = new BigInteger("123");
28     out.println(a.add(b)); // 输出 246
29 }
30
31 static void subtract() {
32     out.println("subtract:");
33     a = new BigInteger("123");
34     b = new BigInteger("123");
35     out.println(a.subtract(b)); // 输出 0
36 }
37
38 static void multiply() {
39     out.println("multiply:");
40     a = new BigInteger("12");
41     b = new BigInteger("12");
42     out.println(a.multiply(b)); // 输出 144
43 }
44
45 static void divide() {
46     out.println("divide:");
47     a = new BigInteger("12");
48     b = new BigInteger("11");
49     out.println(a.divide(b)); // 输出 1
50 }
51
52 static void remainder() {
53     out.println("remainder:");
54     a = new BigInteger("12");
55     b = new BigInteger("10");
56     out.println(a.remainder(b)); // 输出 2
57     a = new BigInteger("-12");
58     b = new BigInteger("10");
59     out.println(a.remainder(b)); // 输出 -2
60 }
61
62 static void mod() {
63     out.println("mod:");
64     a = new BigInteger("12");
65     b = new BigInteger("10");
66     out.println(a.mod(b)); // 输出 2
67     a = new BigInteger("-12");
68     b = new BigInteger("10");
69     out.println(a.mod(b)); // 输出 8
70 }
71
72 static void pow() {
73     out.println("pow:");
74     a = new BigInteger("2");
75     out.println(a.pow(10)); // 输出 1024
76 }
77
78 static void and() {
79     out.println("and:");
80     a = new BigInteger("3"); // 11
81     b = new BigInteger("5"); // 101
82     out.println(a.and(b)); // 输出 1
83 }
84
85 static void or() {
86     out.println("or:");
87     a = new BigInteger("2"); // 10
88     b = new BigInteger("5"); // 101
89     out.println(a.or(b)); // 输出 7
90 }
91
92 static void not() {
93     out.println("not:");
94     a = new BigInteger("2147483647"); // 01111111 11111111 11111111 11111111
95     out.println(a.not()); // 输出 -2147483648 二进制为: 10000000 00000000 00000000 00000000
96 }
97
98 static void xor() {
99     out.println("xor:");
100    a = new BigInteger("6"); // 110
101    b = new BigInteger("5"); // 101
102    out.println(a.xor(b)); // 011 输出 3
103 }
104
105 static void shiftLeft() {
106    out.println("shiftLeft:");
107    a = new BigInteger("1");
108    out.println(a.shiftLeft(10)); // 输出 1024
109 }
110

```

```

111 static void shiftRight() {
112     out.println("shiftRight:");
113     a = new BigInteger("1024");
114     out.println(a.shiftRight(8)); // 输出 4
115 }
116
117 static void max() {
118     out.println("max:");
119     a = new BigInteger("6");
120     b = new BigInteger("5");
121     out.println(a.max(b)); // 输出 6
122 }
123
124 static void min() {
125     out.println("min:");
126     a = new BigInteger("6");
127     b = new BigInteger("5");
128     out.println(a.min(b)); // 输出 5
129 }
130
131 static void bitCount() {
132     out.println("bitCount:");
133     a = new BigInteger("6"); // 110
134     out.println(a.bitCount()); // 输出 2
135 }
136
137 static void bitLength() {
138     out.println("bitLength:");
139     a = new BigInteger("6"); // 110
140     out.println(a.bitLength()); // 输出 3
141 }
142
143 static void getLowestSetBit() {
144     out.println("getLowestSetBit:");
145     a = new BigInteger("8"); // 1000
146     out.println(a.getLowestSetBit()); // 输出 3
147 }
148
149 static void compareTo() {
150     out.println("compareTo:");
151     a = new BigInteger("8");
152     b = new BigInteger("9");
153     out.println(a.compareTo(b)); // 输出 -1
154     a = new BigInteger("8");
155     b = new BigInteger("8");
156     out.println(a.compareTo(b)); // 输出 0
157     a = new BigInteger("8");
158     b = new BigInteger("7");
159     out.println(a.compareTo(b)); // 输出 1
160 }
161
162 static void toStringTest() {
163     out.println("toString:");
164     a = new BigInteger("15");
165     out.println(a.toString()); // 输出 15
166     out.println(a.toString(16)); // 输出 f
167 }
168
169 public static void main(String[] args) {
170     abs();
171     negate();
172     add();
173     subtract();
174     multiply();
175     divide();
176     remainder();
177     mod();
178     pow();
179     and();
180     or();
181     not();
182     xor();
183     shiftLeft();
184     shiftRight();
185     max();
186     min();
187     bitCount();
188     bitLength();
189     getLowestSetBit();
190     compareTo();
191     toStringTest();
192     out.close();
193 }
194 }

```

数学运算

以下均用 `this` 代替当前 `BigInteger` :

函数名	功能
<code>gcd(BigInteger val)</code>	返回 <code>this</code> 的绝对值与 <code>val</code> 的绝对值的最大公约数
<code>isProbablePrime(int val)</code>	返回一个表示 <code>this</code> 是否是素数的布尔值
<code>nextProbablePrime()</code>	返回第一个大于 <code>this</code> 的素数

函数名	功能
<code>modPow(BigInteger b, BigInteger p)</code>	返回 $this^b \bmod p$
<code>modInverse(BigInteger p)</code>	返回 $a \bmod p$ 的乘法逆元

使用案例如下：

```
1 import java.io.PrintWriter;
2 import java.math.BigInteger;
3
4 public class Main {
5     static PrintWriter out = new PrintWriter(System.out);
6     static BigInteger a, b, p;
7
8     static void gcd() { // 最大公约数
9         a = new BigInteger("120032414321432144212100");
10        b = new BigInteger("240231431243123412432140");
11        out.println(String.format("gcd(%s,%s)=%s", a.toString(), b.toString(), a.gcd(b).toString())); // gcd(120032414321432144212100,240231431243123412432140)=20
12    }
13
14    static void isPrime() { // 基于米勒罗宾判定该数是否是素数，参数越大准确性越高，复杂度越高。准确性为 (1-1/(val*2))
15        a = new BigInteger("1200324143214321442127");
16        out.println("a:" + a.toString());
17        out.println(a.isProbablePrime(10) ? "a is prime" : "a is not prime"); // a is not prime
18    }
19
20    static void nextPrime() { // 找出该数的下一个素数
21        a = new BigInteger("1200324143214321442127");
22        out.println("a:" + a.toString());
23        out.println(String.format("a nextPrime is %s", a.nextProbablePrime().toString())); // a nextPrime is 1200324143214321442199
24    }
25
26    static void modPow() { // 快速幂，比正常版本要快，内部有数学优化
27        a = new BigInteger("2");
28        b = new BigInteger("10");
29        p = new BigInteger("1000");
30        out.println(String.format("a:%s b:%s p:%s", a, b, p));
31        out.println(String.format("a^b mod p:%s", a.modPow(b, p).toString())); // 2^10 mod 1000 = 24
32    }
33
34    static void modInverse() { // 逆元
35        a = new BigInteger("10");
36        b = new BigInteger("3");
37        out.println(a.modInverse(b)); // 10^-1 mod 3 = 1
38    }
39
40    public static void main(String[] args) {
41        gcd();
42        isPrime();
43        nextPrime();
44        modPow();
45        modInverse();
46        out.close();
47    }
48 }
```

关于米勒罗宾相关知识可以查阅[Miller-Rabin 素性测试](#)。

基本数据类型与包装数据类型

简介

由于基本类型没有面向对象的特征，为了他们参加到面向对象的开发中，Java 为八个基本类型提供了对应的包装类，分别是 `Byte`、`Double`、`Float`、`Integer`、`Long`、`Short`、`Character` 和 `Boolean`。两者之间的对应关系如下：

基本数据类型 包装数据类型

<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>boolean</code>	<code>Boolean</code>
<code>char</code>	<code>Character</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>

区别

此处以 `int` 与 `Integer` 举例：

- `Integer` 是 `int` 的包装类，`int` 则是 Java 的一种基本类型数据。
- `Integer` 类型实例后才能使用，而 `int` 类型不需要。
- `Integer` 实际对应的引用，当 `new` 一个 `Integer` 时，实际上生成了一个对象，而 `int` 则是直接存储数据。
- `Integer` 的默认值是 `null`，可接受 `null` 和 `int` 类型的数据，`int` 默认值是 `0`，不能接受 `null` 类型的数据。
- `Integer` 判定二个变量是否相同使用 `==` 可能会导致不正确的结果，只能使用 `equals()`，而 `int` 可以直接使用 `==`。

装箱与拆箱

此处以 `int` 与 `Integer` 举例：

`Integer` 的本质是对象，`int` 是基本类型，两个类型之间是不能直接赋值的。需要转换时，应将基础类型转换为包装类型，这种做法称为装箱，反过来则称为拆箱。

```
1 // 基本类型
2 int value1 = 1;
3 // 装箱转换为包装类型
4 Integer integer = Integer.valueOf(value1);
5 // 拆箱转换为基本类型
6 int value2 = integer.intValue();
```

Java 5 引入了自动装箱拆箱机制：

```
1 Integer integer = 1;
2 int value = integer;
```

▼ 注意

虽然 JDK 增加了自动装箱拆箱的机制，但在声明变量时请选择合适的类型，因为包装类型 `Integer` 可以接受 `null`，而基本类型 `int` 不能接受 `null`。因此，对使用 `null` 值的包装类型进行拆箱操作时，会抛出异常。

```
1 Integer integer = Integer.valueOf(null);
2 integer.intValue(); // 抛出 java.lang.NumberFormatException 异常
3
4 Integer integer = null;
5 integer.intValue(); // 抛出 java.lang.NullPointerException 异常
```

继承

基于已有的设计创造新的设计，就是面向对象程序设计中的继承。在继承中，新的类不是凭空产生的，而是基于一个已经存在的类而定义出来的。通过继承，新的类自动获得了基础类中所有的成员，包括成员变量和方法，包括各种访问属性的成员，无论是 `public` 还是 `private`。显然，通过继承来定义新的类，远比从头开始写一个新的类要简单快捷和方便。继承是支持代码重用的重要手段之一。

在 Java 中，继承的关键字为 `extends`，且 Java 只支持单继承，但可以实现多接口。

在 Java 中，所有类都是 `Object` 类的子类。

子类继承父类，所有的父类的成员，包括变量和方法，都成为了子类的成员，除了构造方法。构造方法是父类所独有的，因为它们的名字就是类的名字，所以父类的构造方法在子类中不存在。除此之外，子类继承得到了父类所有的成员。

每个成员有不同的访问属性，子类继承得到了父类所有的成员，但是不同的访问属性使得子类在使用这些成员时有所不同：有些父类的成员直接成为子类的对外的界面，有些则被深深地隐藏起来，即使子类自己也不能直接访问。

下表列出了不同访问属性的父类成员在子类中的访问属性：

父类成员访问属性	在父类中的含义	在子类中的含义
<code>public</code>	对所有类开放	对所有类开放
<code>protected</code>	只有包内其它类、自己和子类可以访问	只有包内其它类、自己和子类可以访问
缺省（default）	只有包内其它类可以访问	如果子类与父类在同一个包内，只有包内其它类可以访问；否则相当于 <code>private</code> ，不能访问
<code>private</code>	只有自己可以访问	不能访问

多态

在 Java 中当把一个对象赋值给一个变量时，对象的类型必须与变量的类型相匹配。但由于 Java 有继承的概念，便可重新定义为一个变量可以保存其所声明的类型或该类型的任何子类型。

如果一个类型实现了接口，也可以称之为该接口的子类型。

Java 中保存对象类型的变量是多态变量。「多态」这个术语（字面意思是许多形态）是指一个变量可以保存不同类型（即其声明的类型或任何子类型）的对象。

多态变量：

1. Java 的对象变量是多态的，它们能保存不止一种类型的对象。
2. 它们可以保存的是声明类型的对象，或声明类型子类的对象。
3. 当把子类的对象赋给父类的变量的时候，就发生了向上转型。

泛型

泛型指在类定义时不设置类中的属性或方法参数的具体类型，而是在使用（或创建对象）时再进行类型的定义。泛型本质是参数化类型，即所操作的数据类型被指定为一个参数。

泛型提供了编译时类型安全检测的机制，该机制允许编译时检测非法类型。

接口

简介

接口（英文：Interface）在 Java 中是一个抽象类型，是抽象方法的集合，通常以 `interface` 来声明。一个类通过实现接口的方式，从而来继承接口的抽象方法。

接口并不是类，编写接口的方式和类很相似，但是它们属于不同的概念。类描述对象的属性和方法。接口则包含类要实现的方法。

除非实现接口的类是抽象类，否则该类要定义接口中的所有方法。

接口无法被实例化，但是可以被实现。一个实现接口的类，必须实现接口内所描述的所有方法，否则就必须声明为抽象类。另外，在 Java 中，接口类型可用来声明一个变量，他们可以成为一个空指针，或是被绑定在一个以此接口实现的对象。

与类的区别

1. 接口不能用于实例化对象。
2. 接口没有构造方法。
3. 接口中所有的方法必须是抽象方法，Java 8 之后接口中可以使用 `default` 关键字修饰的非抽象方法。
4. 接口不能包含成员变量，除了 `static` 和 `final` 变量。
5. 接口不是被类继承了，而是要被类实现。
6. 接口支持多继承，类不支持多继承。

声明

```
1 [可见度] interface 接口名称 [extends 其他的接口名] {
2     // 声明变量
3     // 抽象方法
4 }
```

实现

```
1 ...implements 接口名称[, 其他接口名称, 其他接口名称..., ...] ...
```

Lambda 表达式

简介

lambda 表达式也可称为闭包，是 Java 8 的最重要的新特性。

lambda 表达式允许把函数作为一个方法的参数（函数作为参数传递进方法中）。

使用 lambda 表达式可以使代码变的更加简洁紧凑。

语法

可选类型声明：不需要声明参数类型，编译器可以统一识别参数值。

可选的参数圆括号：一个参数无需定义圆括号，但多个参数需要定义圆括号。

可选的大括号：如果主体包含了一个语句，就不需要使用大括号。

可选的返回关键字：如果主体只有一个表达式返回值则编译器会自动返回值，大括号需要指定表达式返回了一个数值。

lambda 表达式声明方式如下：

以字符串数组按长度排序的自定义比较器为例：

1. 参数，箭头，一个表达式。

```
1 import java.util.Arrays;
2
3 public class Main {
4     static PrintWriter out = new PrintWriter(System.out);
5
6     public static void main(String[] args) {
7         String[] plants = {"Mercury", "venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"};
8         Arrays.sort(plants, (String first, String second) -> (first.length() - second.length()));
9         for (String word : plants) {
10             out.print(word + " ");
11         }
12         out.close();
13     }
14 }
```

2. 参数，箭头，多条语句。

```
1 import java.io.PrintWriter;
2 import java.util.Arrays;
3
4 public class Main {
5     static PrintWriter out = new PrintWriter(System.out);
6
7     public static void main(String[] args) {
8         String[] plants = {"Mercury", "venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"};
9         Arrays.sort(plants, (first, second) ->
10             {
11                 // 形参不写类型, 可以从上下文判断出
12                 int result = first.length() - second.length();
13                 return result;
14             });
15         for (String word : plants) {
16             out.print(word + " ");
17         }
18         out.close();
19     }
20 }
```

-> 是一个推导符号，表示前面的括号接收到参数，推导后面的返回值（其实就是传递了方法）。

1. 常用形式：

```

1 // 1. 不需要参数, 返回值为 5
2 () -> 5
3
4 // 2. 接收一个参数 (数字类型), 返回其 2 倍的值
5 x -> 2 * x
6
7 // 3. 接受 2 个参数 (数字) 并返回他们的差值
8 (x, y) -> x - y
9
10 // 4. 接收 2 个 int 类型整数并返回他们的和
11 (int x, int y) -> x + y
12
13 // 5. 接受一个 String 对象并在控制台打印, 不返回任何值 (看起来像是返回 void)
14 (String s) -> System.out.print(s)

```

函数式接口

1. 是一个接口, 符合 Java 接口定义。
2. 只包含一个抽象方法的接口。
3. 因为只有一个未实现的方法, 所以 lambda 表达式可以自动填上去。

函数式接口使用方式如下: 1. 输出长度为 2 的倍数的字符串。

```

1 import java.io.PrintWriter;
2
3 public class Main {
4     static PrintWriter out = new PrintWriter(System.out);
5
6     public static void main(String[] args) {
7         String[] plants = {"Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune"};
8         Test test = s -> { // lambda 表达式作为函数式接口的实例
9             if (s.length() % 2 == 0) {
10                 return true;
11             }
12             return false;
13         };
14         for (String word : plants) {
15             if (test.check(word)) {
16                 out.print(word + " ");
17             }
18         }
19         out.close();
20     }
21 }
22
23 interface Test {
24     public boolean check(String s);
25 }

```

2. 实现加减乘除四则运算。

```

1 import java.io.PrintWriter;
2
3 public class Main {
4     static PrintWriter out = new PrintWriter(System.out);
5
6     public static double calc(double a, double b, Calculator util) {
7         return util.operation(a, b);
8     }
9
10    public static void main(String[] args) {
11        Calculator util[] = new Calculator[4]; // 定义函数式接口数组
12        util[0] = (a, b) -> a + b;
13        util[1] = (a, b) -> a - b;
14        util[2] = (a, b) -> a * b;
15        util[3] = (a, b) -> a / b;
16        double a = 20, b = 15;
17        for (Calculator c : util) {
18            System.out.println(calc(a, b, c));
19        }
20        out.close();
21    }
22 }
23
24 interface Calculator {
25     public double operation(double a, double b);
26 }

```

Collection

Collection 是 Java 中的接口, 被多个泛型容器接口所实现。在这里, Collection 是指代存放对象类型的数据结构。

Java 中的 Collection 元素类型定义时必须为对象, 不能为基本数据类型。

以下内容用法均基于 Java 里多态的性质, 均是以实现接口的形式出现。

常用的接口包括 List、Queue、Set 和 Map。

容器定义

1. 当定义泛型容器类时, 需要在定义时指定数据类型。

例如：

```
1 List<Integer> list1 = new LinkedList<>();
```

- 1. 倘若不指定数据类型，而当成 `Object` 类型随意添加数据，在 `Java 8` 中虽能编译通过，但会有很多警告风险。

例如：

```
1 List list = new ArrayList<>();
2 list.add(1);
3 list.add(true);
4 list.add(1.01);
5 list.add(1L);
6 list.add("I am String");
```

因此，如果没有特殊需求的话不推荐第 2 种行为，编译器无法帮忙检查存入的数据是否安全。`list.get(index)` 取值时无法明确数据的类型（取到的数据类型都为 `Object`），需要手动转回原来的类型，稍有不慎可能出现误转型异常。

如果是明确了类型如 `List<LTInteger>`，此时编译器会检查放入的数据类型，只能放入整数的数据。声明集合变量时只能使用包装类型 `List<LTInteger>` 或者自定义的 `Class`，而不能是基本类型如 `List<LTint>`。

List

ArrayList

`ArrayList` 是支持可以根据需求动态生长的数组，初始长度默认为 10。如果超出当前长度便扩容。

初始化

```
1 import java.io.PrintWriter;
2 import java.util.ArrayList;
3 import java.util.List;
4
5 public class Main {
6     static PrintWriter out = new PrintWriter(System.out);
7
8     public static void main(String[] args) {
9         List<Integer> list1 = new ArrayList<>(); // 创建一个名字为 list1 的可自增数组，初始长度为默认值（10）
10        List<Integer> list2 = new ArrayList<>(30); // 创建一个名字为list2的可自增数组，初始长度为 30
11        List<Integer> list3 = new ArrayList<>(list2); // 创建一个名字为 list3 的可自增数组，使用 list2 里的元素和 size 作为自己的初始值
12    }
13 }
```

LinkedList

`LinkedList` 是双链表。

初始化

```
1 import java.io.PrintWriter;
2 import java.util.LinkedList;
3 import java.util.List;
4
5 public class Main {
6     static PrintWriter out = new PrintWriter(System.out);
7
8     public static void main(String[] args) {
9         List<Integer> list1 = new LinkedList<>(); // 创建一个名字为 list1 的双链表
10        List<Integer> list2 = new LinkedList<>(list1); // 创建一个名字为 list2 的双链表，将 list1 内所有元素加入进来
11    }
12 }
```

常用方法

以下均用 `this` 代替当前 `List<LTInteger>`：

函数名	功能
<code>size()</code>	返回 <code>this</code> 的长度
<code>add(Integer val)</code>	在 <code>this</code> 尾部插入一个元素
<code>add(int idx, Integer e)</code>	在 <code>this</code> 指定位置插入一个元素
<code>get(int idx)</code>	返回 <code>this</code> 中第 <code>idx</code> 位置的值，若越界则抛出异常
<code>set(int idx, Integer e)</code>	修改 <code>this</code> 中第 <code>idx</code> 位置的值

使用案例及区别对比：

```

1 import java.io.PrintWriter;
2 import java.util.ArrayList;
3 import java.util.LinkedList;
4 import java.util.List;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static List<Integer> array = new ArrayList<>();
9     static List<Integer> linked = new LinkedList<>();
10
11     static void add() {
12         array.add(1); // 时间复杂度为 O(1)
13         linked.add(1); // 时间复杂度为 O(1)
14     }
15
16     static void get() {
17         array.get(10); // 时间复杂度为 O(1)
18         linked.get(10); // 时间复杂度为 O(11)
19     }
20
21     static void addIdx() {
22         array.add(0, 2); // 最坏情况下时间复杂度为 O(n)
23         linked.add(0, 2); // 最坏情况下时间复杂度为 O(n)
24     }
25
26     static void size() {
27         array.size(); // 时间复杂度为 O(1)
28         linked.size(); // 时间复杂度为 O(1)
29     }
30
31     static void set() { // 该方法返回值为原本该位置元素的值
32         array.set(0, 1); // 时间复杂度为 O(1)
33         linked.set(0, 1); // 最坏时间复杂度为 O(n)
34     }
35
36 }

```

遍历

```

1 import java.io.PrintWriter;
2 import java.util.ArrayList;
3 import java.util.Iterator;
4 import java.util.LinkedList;
5 import java.util.List;
6
7 public class Main {
8     static PrintWriter out = new PrintWriter(System.out);
9     static List<Integer> array = new ArrayList<>();
10    static List<Integer> linked = new LinkedList<>();
11
12    static void function1() { // 朴素遍历
13        for (int i = 0; i < array.size(); i++) {
14            out.println(array.get(i)); // 遍历自增数组, 复杂度为 O(n)
15        }
16        for (int i = 0; i < linked.size(); i++) {
17            out.println(linked.get(i)); // 遍历双链表, 复杂度为 O(n^2), 因为 LinkedList 的 get(i) 复杂度是 O(i)
18        }
19    }
20
21    static void function2() { // 增强 for 循环遍历
22        for (int e : array) {
23            out.println(e);
24        }
25        for (int e : linked) {
26            out.println(e); // 复杂度均为 O(n)
27        }
28    }
29
30    static void function3() { // 迭代器遍历
31        Iterator<Integer> iterator1 = array.iterator();
32        Iterator<Integer> iterator2 = linked.iterator();
33        while (iterator1.hasNext()) {
34            out.println(iterator1.next());
35        }
36        while (iterator2.hasNext()) {
37            out.println(iterator2.next());
38        } // 复杂度均为 O(n)
39    }
40
41 }

```

▼ 注意

不要在 `for/foreach` 遍历 `List` 的过程中删除其中的元素, 否则会抛出异常。

原因也很简单, `list.size()` 改变了, 但在循环中已循环的次数却是没有随之变化。原来预计在下一个 `index` 的数据因为删除的操作变成了当前 `index` 的数据, 运行下一个循环时操作的会变为原来预计在下下个 `index` 的数据, 最终会导致操作的数据不符合预期。

Queue

LinkedList

可以使用 `LinkedList` 实现普通队列, 底层是链表模拟队列。

初始化

```
1 Queue<Integer> q = new LinkedList<>();
```

LinkedList 底层实现了 List 接口与 Deque 接口，而 Deque 接口继承自 Queue 接口，所以 LinkedList 可以同时实现 List 与 Queue 。

ArrayDeque

可以使用 ArrayDeque 实现普通队列，底层是数组模拟队列。

初始化

```
1 Queue<Integer> q = new ArrayDeque<>();
```

ArrayDeque 底层实现了 Deque 接口，而 Deque 接口继承自 Queue 接口，所以 ArrayDeque 可以实现 Queue。

LinkedList 与 ArrayDeque 在实现 Queue 接口上的区别

- 1. 数据结构：在数据结构上，ArrayDeque 和 LinkedList 都实现了 Java Deque 双端队列接口。但 ArrayDeque 没有实现了 Java List 列表接口，所以不具备根据索引位置操作的行为。
- 2. 线程安全：ArrayDeque 和 LinkedList 都不考虑线程同步，不保证线程安全。
- 3. 底层实现：在底层实现上，ArrayDeque 是基于动态数组的，而 LinkedList 是基于双向链表的。
- 4. 在遍历速度上：ArrayDeque 是一块连续内存空间，基于局部性原理能够更好地命中 CPU 缓存行，而 LinkedList 是离散的内存空间对缓存行不友好。
- 5. 在操作速度上：ArrayDeque 和 LinkedList 的栈和队列行为都是 时间复杂度，ArrayDeque 的入栈和入队有可能会触发扩容，但从均摊分析上看依然是 时间复杂度。
- 6. 额外内存消耗上：ArrayDeque 在数组的头指针和尾指针外部有闲置空间，而 LinkedList 在节点上增加了前驱和后继指针。

PriorityQueue

PriorityQueue 是优先队列，默认是小根堆。

初始化

```
1 Queue<Integer> q1 = new PriorityQueue<>(); // 小根堆
2 Queue<Integer> q2 = new PriorityQueue<>((x, y) -> {return y - x;}); // 大根堆
```

常用方法

以下均用 this 代替当前 Queue<Integer>：

函数名	功能
size()	返回 this 的长度
add(Integer val)	入队
offer(Integer val)	入队
isEmpty()	判断队列是否为空，为空则返回 true
peek()	返回队头元素
poll()	返回队头元素并删除

使用案例及区别对比：

```

1 import java.io.PrintWriter;
2 import java.util.LinkedList;
3 import java.util.PriorityQueue;
4 import java.util.Queue;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static Queue<Integer> q1 = new LinkedList<>();
9     static Queue<Integer> q2 = new PriorityQueue<>();
10
11     static void add() { // add 和 offer 功能上没有差距，区别是是否会抛出异常
12         q1.add(1); // 时间复杂度为 O(1)
13         q2.add(1); // 时间复杂度为 O(logn)
14     }
15
16     static void isEmpty() {
17         q1.isEmpty(); // 时间复杂度为 O(1)
18         q2.isEmpty(); // 空间复杂度为 O(1)
19     }
20
21     static void size() {
22         q1.size(); // 时间复杂度为 O(1)
23         q2.size(); // 返回 q2 的长度
24     }
25
26     static void peek() {
27         q1.peek(); // 时间复杂度为 O(1)
28         q2.peek(); // 时间复杂度为 O(logn)
29     }
30
31     static void poll() {
32         q1.poll(); // 时间复杂度为 O(1)
33         q2.poll(); // 时间复杂度为 O(logn)
34     }
35 }

```

遍历

```

1 import java.io.PrintWriter;
2 import java.util.LinkedList;
3 import java.util.PriorityQueue;
4 import java.util.Queue;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static Queue<Integer> q1 = new LinkedList<>();
9     static Queue<Integer> q2 = new PriorityQueue<>();
10
11     static void test() {
12         while (!q1.isEmpty()) { // 复杂度为 O(n)
13             out.println(q1.poll());
14         }
15         while (!q2.isEmpty()) { // 复杂度为 O(nlogn)
16             out.println(q2.poll());
17         }
18     }
19 }
20 }

```

Deque

Deque 是 Java 中的双端队列，我们通常用其进行队列的操作以及栈的操作。

主要函数

以下均用 this 代替当前 Deque<Integer>：

函数名	功能
push(Integer val)	将一个元素从队头加入this，等效于addFirst
pop()	将队头元素删除，等效于removeFirst
addFirst(Integer val)	将一个元素从队头加入this
removeFirst()	将队头元素删除，并返回该元素
addLast(Integer val)	将一个元素从队尾加入this
removeLast()	将队尾元素删除，并返回该元素
offerFirst(Integer val)	将一个元素从队头加入this
pollFirst()	将队头元素删除，并返回该元素
offerLast(Integer val)	将一个元素从队尾加入this
pollLast()	将队尾元素删除，并返回该元素
add(Integer val)	将一个元素从队尾加入this
offer(Integer val)	将一个元素从队尾加入this
poll()	将队头元素删除，并返回该元素
remove()	将队头元素删除，并返回该元素
peekFirst()	返回队头元素
peekLast()	返回队尾元素

add、remove 操作在遇到异常时会抛出异常，而offer、poll 不会抛出异常。

栈的操作

```
1 import java.util.ArrayDeque;
2 import java.util.Deque;
3
4 public class Main {
5     static Deque<Integer> stack = new ArrayDeque<>();
6     static int[] a = {1, 2, 3, 4, 5};
7
8     public static void main(String[] args) {
9         for (int v : a) {
10             stack.push(v);
11         }
12         while (!stack.isEmpty()) { //输出 5 4 3 2 1
13             System.out.println(stack.pop());
14         }
15     }
16 }
```

双端队列的操作

```
1 import java.util.ArrayDeque;
2 import java.util.Deque;
3
4 public class Main {
5     static Deque<Integer> deque = new ArrayDeque<>();
6
7     static void insert() {
8         deque.addFirst(1);
9         deque.addFirst(2);
10        deque.addLast(3);
11        deque.addLast(4);
12    }
13
14    public static void main(String[] args) {
15        insert();
16        while (!deque.isEmpty()) { //输出 2 1 3 4
17            System.out.println(deque.poll());
18        }
19        insert();
20        while (!deque.isEmpty()) { //输出 4 3 1 2
21            System.out.println(deque.pollLast());
22        }
23    }
24 }
```

Set

Set 是保持容器中的元素不重复的一种数据结构。

HashSet

随机位置插入的 Set。

初始化

```
1 Set<Integer> s1 = new HashSet<>();
```

LinkedHashSet

保持插入顺序的 Set。

初始化

```
1 Set<Integer> s2 = new LinkedHashSet<>();
```

TreeSet

保持容器中元素有序的 Set，默认为升序。

初始化

```
1 Set<Integer> s3 = new TreeSet<>();
2 Set<Integer> s4 = new TreeSet<>((x, y) -> {return y - x;}); // 降序
```

TreeSet的更多使用

这些方法是TreeSet新创建并实现的，我们无法使用Set接口调用以下方法，因此我们创建方式如下：

```
1 TreeSet<Integer> s3 = new TreeSet<>();
2 TreeSet<Integer> s4 = new TreeSet<>((x, y) -> {return y - x;}); // 降序
```

以下均用 `this` 代替当前 `TreeSet<Integer>`：

函数名	功能
<code>first()</code>	返回 <code>this</code> 中第一个元素，无则返回 <code>null</code>
<code>last()</code>	返回 <code>this</code> 中最后一个元素，无则返回 <code>null</code>
<code>floor(Integer val)</code>	返回集合中 <code><=val</code> 的第一个元素，无则返回 <code>null</code>
<code>ceiling(Integer val)</code>	返回集合中 <code>>=val</code> 的第一个元素，无则返回 <code>null</code>
<code>higher(Integer val)</code>	返回集合中 <code>>val</code> 的第一个元素，无则返回 <code>null</code>
<code>lower(Integer val)</code>	返回集合中 <code>&LTval</code> 的第一个元素，无则返回 <code>null</code>
<code>pollFirst()</code>	返回并删除 <code>this</code> 中第一个元素，无则返回 <code>null</code>
<code>pollLast()</code>	返回并删除 <code>this</code> 中最后一个元素，无则返回 <code>null</code>

代码示例：

```
1 import java.util.TreeSet;
2
3 public class Main {
4     static int[] a = {4,7,1,2,3,6};
5
6     public static void main(String[] args) {
7         TreeSet<Integer> set = new TreeSet<>();
8         for(int v:a) {
9             set.add(v);
10        }
11        Integer a2 = set.first();
12        System.out.println(a2); //返回 1
13        Integer a3 = set.last();
14        System.out.println(a3); //返回 7
15        Integer a4 = set.floor(5);
16        System.out.println(a4); //返回 4
17        Integer a5 = set.ceiling(6);
18        System.out.println(a5); //返回 6
19        Integer a6 = set.higher(7);
20        System.out.println(a6); //返回 null
21        Integer a7 = set.lower(2);
22        System.out.println(a7); //返回 1
23        Integer a8 = set.pollFirst();
24        System.out.println(a8); //返回 1
25        Integer a9 = set.pollLast();
26        System.out.println(a9); //返回 7
27    }
28 }
```

Set常用方法

以下均用 `this` 代替当前 `Set<Integer>`：

函数名	功能
<code>size()</code>	返回 <code>this</code> 的长度
<code>add(Integer val)</code>	插入一个元素进 <code>this</code>
<code>contains(Integer val)</code>	判断 <code>this</code> 中是否有元素 <code>val</code>
<code>addAll(Collection e)</code>	将一个容器里的所有元素添加进 <code>this</code>
<code>retainAll(Collection e)</code>	将 <code>this</code> 改为两个容器内相同的元素
<code>removeAll(Collection e)</code>	将 <code>this</code> 中与 <code>e</code> 相同的元素删除

使用案例：求并集、交集、差集。

```

1 import java.io.PrintWriter;
2 import java.util.HashSet;
3 import java.util.LinkedHashSet;
4 import java.util.Set;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static Set<Integer> s1 = new HashSet<>();
9     static Set<Integer> s2 = new LinkedHashSet<>();
10
11     static void add() {
12         s1.add(1);
13     }
14
15     static void contains() { // 判断 set 中是否有元素值为 2, 有则返回 true, 否则返回 false
16         s1.contains(2);
17     }
18
19     static void test1() { // s1 与 s2 的并集
20         Set<Integer> res = new HashSet<>();
21         res.addAll(s1);
22         res.addAll(s2);
23     }
24
25     static void test2() { // s1 与 s2 的交集
26         Set<Integer> res = new HashSet<>();
27         res.addAll(s1);
28         res.retainAll(s2);
29     }
30
31     static void test3() { // 差集: s1 - s2
32         Set<Integer> res = new HashSet<>();
33         res.addAll(s1);
34         res.removeAll(s2);
35     }
36 }

```

遍历

```

1 import java.io.PrintWriter;
2 import java.util.HashSet;
3 import java.util.LinkedHashSet;
4 import java.util.Set;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static Set<Integer> s1 = new HashSet<>();
9     static Set<Integer> s2 = new LinkedHashSet<>();
10
11     static void test() {
12         for (int key : s1) {
13             out.println(key);
14         }
15         out.close();
16     }
17 }

```

Map

Map 是维护键值对 <Key, Value> 的一种数据结构, 其中 Key 唯一。

HashMap

随机位置插入的 Map。

初始化

```
1 Map<Integer, Integer> map1 = new HashMap<>();
```

LinkedHashMap

保持插入顺序的 Map。

初始化

```
1 Map<Integer, Integer> map2 = new LinkedHashMap<>();
```

TreeMap

保持 key 有序的 Map, 默认升序。

初始化

```
1 Map<Integer, Integer> map3 = new TreeMap<>();
2 Map<Integer, Integer> map4 = new TreeMap<>((x, y) -> {return y - x;}); // 降序
```

常用方法

以下均用 `this` 代替当前 `Map<Integer, Integer>`:

函数名	功能
<code>put(Integer key, Integer value)</code>	插入一个元素进 <code>this</code>
<code>size()</code>	返回 <code>this</code> 的长度
<code>containsKey(Integer val)</code>	判断 <code>this</code> 中是否有元素 <code>key</code> 为 <code>val</code>
<code>get(Integer key)</code>	将 <code>this</code> 中对应的 <code>key</code> 的 <code>value</code> 返回
<code>keySet</code>	将 <code>this</code> 中所有元素的 <code>key</code> 作为集合返回

使用案例:

```
1 import java.io.PrintWriter;
2 import java.util.HashMap;
3 import java.util.LinkedHashMap;
4 import java.util.Map;
5 import java.util.TreeMap;
6
7 public class Main {
8     static PrintWriter out = new PrintWriter(System.out);
9
10    static Map<Integer, Integer> map1 = new HashMap<>();
11    static Map<Integer, Integer> map2 = new LinkedHashMap<>();
12    static Map<Integer, Integer> map3 = new TreeMap<>();
13    static Map<Integer, Integer> map4 = new TreeMap<>((x,y)->{return y-x;});
14
15    static void put(){ // 将 key 为 1、value 为 1 的元素返回
16        map1.put(1, 1);
17    }
18    static void get(){ // 将 key 为 1 的 value 返回
19        map1.get(1);
20    }
21    static void containsKey(){ // 判断是否有 key 为 1 的键值对
22        map1.containsKey(1);
23    }
24    static void KeySet(){
25        map1.keySet();
26    }
27 }
```

遍历

```
1 import java.io.PrintWriter;
2 import java.util.HashMap;
3 import java.util.Map;
4
5 public class Main {
6     static PrintWriter out = new PrintWriter(System.out);
7
8     static Map<Integer, Integer> map1 = new HashMap<>();
9
10    static void print() {
11        for (int key : map1.keySet()) {
12            out.println(key + " " + map1.get(key));
13        }
14    }
15 }
```

当然，在面向对象的世界里，你的参数是啥都可以，包括 `Collection` 与自定义类型。

例如 `Map` 也可以定义为：

```
1 Map<String, Set<Integer>> map = new HashMap<>();
```

Arrays

`Arrays` 是 `java.util` 中对数组操作的一个工具类。方法均为静态方法，可使用类名直接调用。

`Arrays.sort()`

`Arrays.sort()` 是对数组进行的排序的方法，主要重载方法如下：


```

1 import java.util.Arrays;
2 import java.util.Comparator;
3
4 public class Main {
5     static int[] a = new int[10];
6     static Integer[] b = new Integer[10];
7     static int firstIdx, lastIdx;
8
9     public static void main(String[] args) {
10         Arrays.sort(a); // 1
11         Arrays.sort(a, firstIdx, lastIdx); // 2
12         Arrays.sort(b, new Comparator<Integer>() { // 3
13             @Override
14             public int compare(Integer o1, Integer o2) {
15                 return o2 - o1;
16             }
17         });
18         Arrays.sort(b, firstIdx, lastIdx, new Comparator<Integer>() { // 4
19             @Override
20             public int compare(Integer o1, Integer o2) {
21                 return o2 - o1;
22             }
23         });
24         // 由于 Java 8 后有 Lambda 表达式, 第三个重载及第四个重载亦可写为
25         Arrays.sort(b, (x, y) -> { // 5
26             return y - x;
27         });
28         Arrays.sort(b, (x, y) -> { // 6
29             return y - x;
30         });
31     }
32 }

```

序号所对应的重载方法含义：

1. 对数组 **a** 进行排序，默认升序。
2. 对数组 **a** 的指定位置进行排序，默认升序，排序区间为左闭右开 `[firstIdx, lastIdx)`。
3. 对数组 **a** 以自定义的形式排序，第二个参数 - 第一个参数为降序, 第一个参数 - 第二个参数为升序，当自定义排序比较器时，数组元素类型必须为对象类型。
4. 对数组 **a** 的指定位置进行自定义排序，排序区间为左闭右开 `[firstIdx, lastIdx)`，当自定义排序比较器时，数组元素类型必须为对象类型。
5. 和 3 同理，用 Lambda 表达式优化了代码长度。
6. 和 4 同理，用 Lambda 表达式优化了代码长度。

Arrays.sort() 底层函数：

1. 当你 `Arrays.sort` 的参数数组元素类型为基本数据类型（`byte`、`short`、`char`、`int`、`long`、`double`、`float`）时，默认为 `DualPivotQuicksort`（双轴快排），复杂度最坏可以达到 $O(n^2)$ 。
2. 当你 `Arrays.sort` 的参数数组元素类型为非基本数据类型时，则默认为 `legacyMergeSort` 和 `TimSort`（归并排序），复杂度为 $O(n \log n)$ 。

可以通过如下代码验证：

▼ [Codeforces 1646B - Quality vs Quantity](#)

题意概要：有 n 个数，你需要将其分为 2 组，是否存在 1 组的长度小于另 1 组的同时和大于它。

► 例题代码

如果你将以上代码的 `a` 数组类型由 `Integer` 修改为 `int`，会导致 TLE。

Arrays.binarySearch()

`Arrays.binarySearch()` 是对数组连续区间进行二分搜索的方法，前提是数组必须有序，时间复杂度为 $O(\log n)$ ，主要重载方法如下：

```

1 import java.util.Arrays;
2
3 public class Main {
4     static int[] a = new int[10];
5     static Integer[] b = new Integer[10];
6     static int firstIdx, lastIdx;
7     static int key;
8
9     public static void main(String[] args) {
10         Arrays.binarySearch(a, key); // 1
11         Arrays.binarySearch(a, firstIdx, lastIdx, key); // 2
12     }
13 }

```

源码如下：

```

1 private static int binarySearch0(int[] a, int fromIndex, int toIndex, int key) {
2     int low = fromIndex;
3     int high = toIndex - 1;
4
5     while (low <= high) {
6         int mid = (low + high) >>> 1;
7         int midVal = a[mid];
8
9         if (midVal < key)
10             low = mid + 1;
11         else if (midVal > key)
12             high = mid - 1;
13         else
14             return mid; // key found
15     }
16     return -(low + 1); // key not found.
17 }

```

序号所对应的重载方法含义：

1. 从数组 **a** 中二分查找是否存在 **key**，如果存在，便返回其下标。若不存在，则返回一个负数。
2. 从数组 **a** 中二分查找是否存在 **key**，如果存在，便返回其下标,搜索区间为左闭右开 `[firstIdx,lastIdx)`。若不存在，则返回一个负数。

Arrays.fill()

`Arrays.fill()` 方法将数组中连续位置的元素赋值为统一元素。其接受的参数为数组、`fromIndex`、`toIndex` 和需要填充的数。方法执行后，数组左闭右开区间 `[firstIdx,lastIdx)` 内的所有元素的值均为需要填充的数。

Collections

`Collections` 是 `java.util` 中对集合操作的一个工具类。方法均为静态方法，可使用类名直接调用。

Collections.sort()

`Collections.sort()` 底层原理为将其中所有元素转化为数组调用 `Arrays.sort()`，完成排序后再赋值给原本的集合。又因为 Java 中 `Collection` 的元素类型均为对象类型，所以始终是归并排序去处理。

该方法无法对集合指定区间排序。

底层源码：

```

1 default void sort(Comparator<? super E> c) {
2     Object[] a = this.toArray();
3     Arrays.sort(a, (Comparator) c);
4     ListIterator<E> i = this.listIterator();
5     for (Object e : a) {
6         i.next();
7         i.set((E) e);
8     }
9 }

```

Collections.binarySearch()

`Collections.binarySearch()` 是对集合中指定区间进行二分搜索,功能与 `Arrays.binarySearch()` 相同。

```
1 Collections.binarySearch(list, key);
```

该方法无法对指定区间进行搜索。

Collections.swap()

`Collections.swap()` 的功能是交换集合中指定二个位置的元素。

```
1 Collections.swap(list, i, j);
```

其他

1. -0.0 != 0.0

在 Java 中，如果单纯是数值类型，`-0.0 = 0.0`。若是对象类型，则 `-0.0 != 0.0`。倘若你尝试用 `Set` 统计斜率数量时，这个问题就会带来麻烦。提供的解决方式是在所有的斜率加入 `Set` 前将值增加 `0.0`。

```
1 import java.io.PrintWriter;
2
3 public class Main {
4     static PrintWriter out = new PrintWriter(System.out);
5
6     static void A() {
7         Double a = 0.0;
8         Double b = -0.0;
9         out.println(a.equals(b)); // false
10    }
11
12    static void B() {
13        Double a = 0.0;
14        Double b = -0.0 + 0.0;
15        out.println(a.equals(b)); // true
16    }
17
18    static void C() {
19        double a = 0.0;
20        double b = -0.0;
21        out.println(a == b); // true
22    }
23
24
25    public static void main(String[] args) {
26        A();
27        B();
28        C();
29        out.close();
30    }
31 }
```

参考资料

1. [Input & Output - USACO Guide ↗](#)
-

本页面最近更新：2023/10/10 23:17:11，[更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[shuzhouliu](#), [1804040636](#), [aofall](#), [caopengrun](#), [CCXXI](#), [Cubik65536](#), [ImpleLee](#), [megakite](#), [optimize-2](#), [Tiphereth-A](#), [untitledunrevised](#), [yusancky](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用